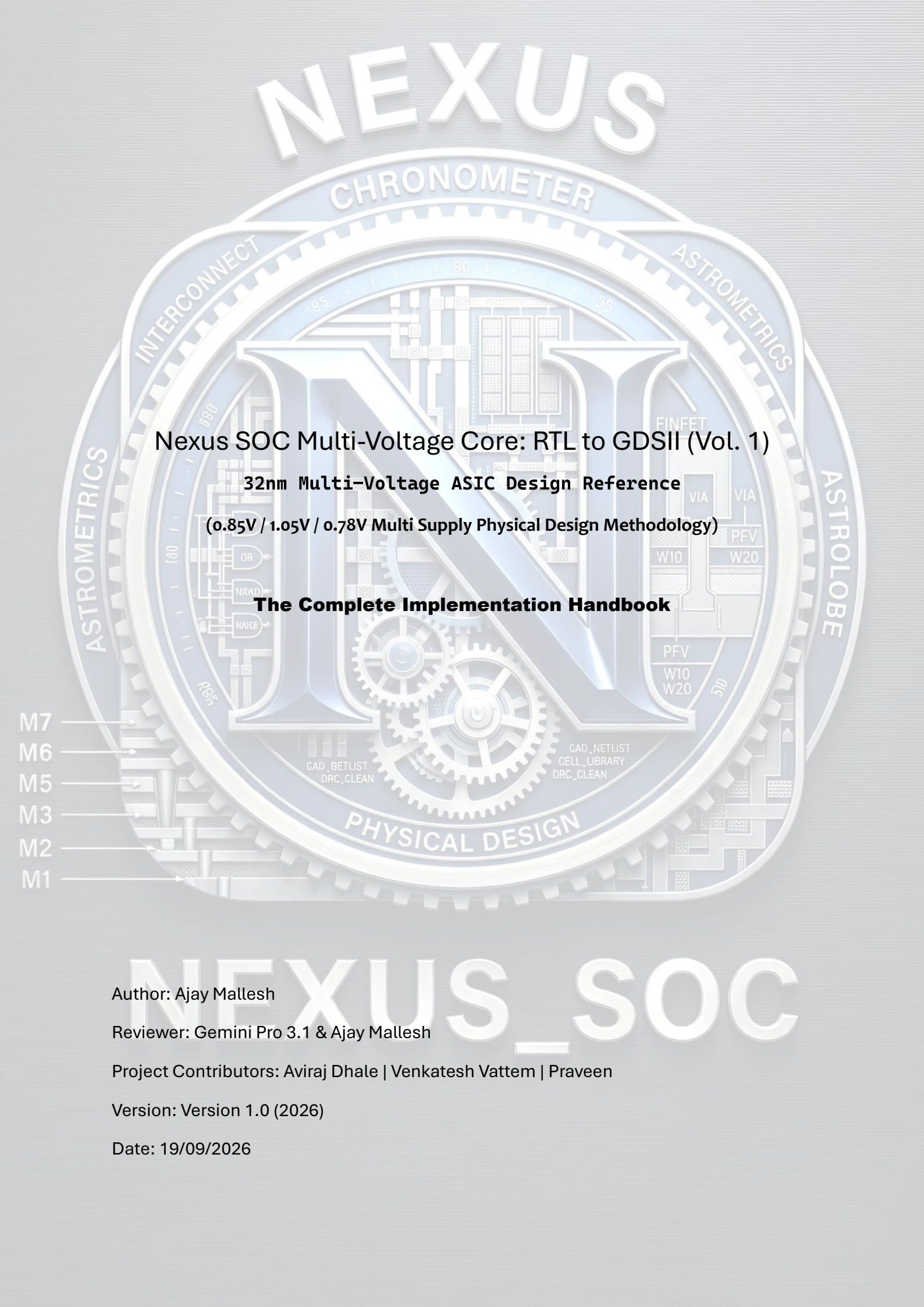



# Nexus SOC Multi-Voltage Core: RTL to GDSII (Vol. 1)

**32nm Multi-Voltage ASIC Design Reference**

**(0.85V / 1.05V / 0.78V Multi Supply Physical Design Methodology)**

## The Complete Implementation Handbook

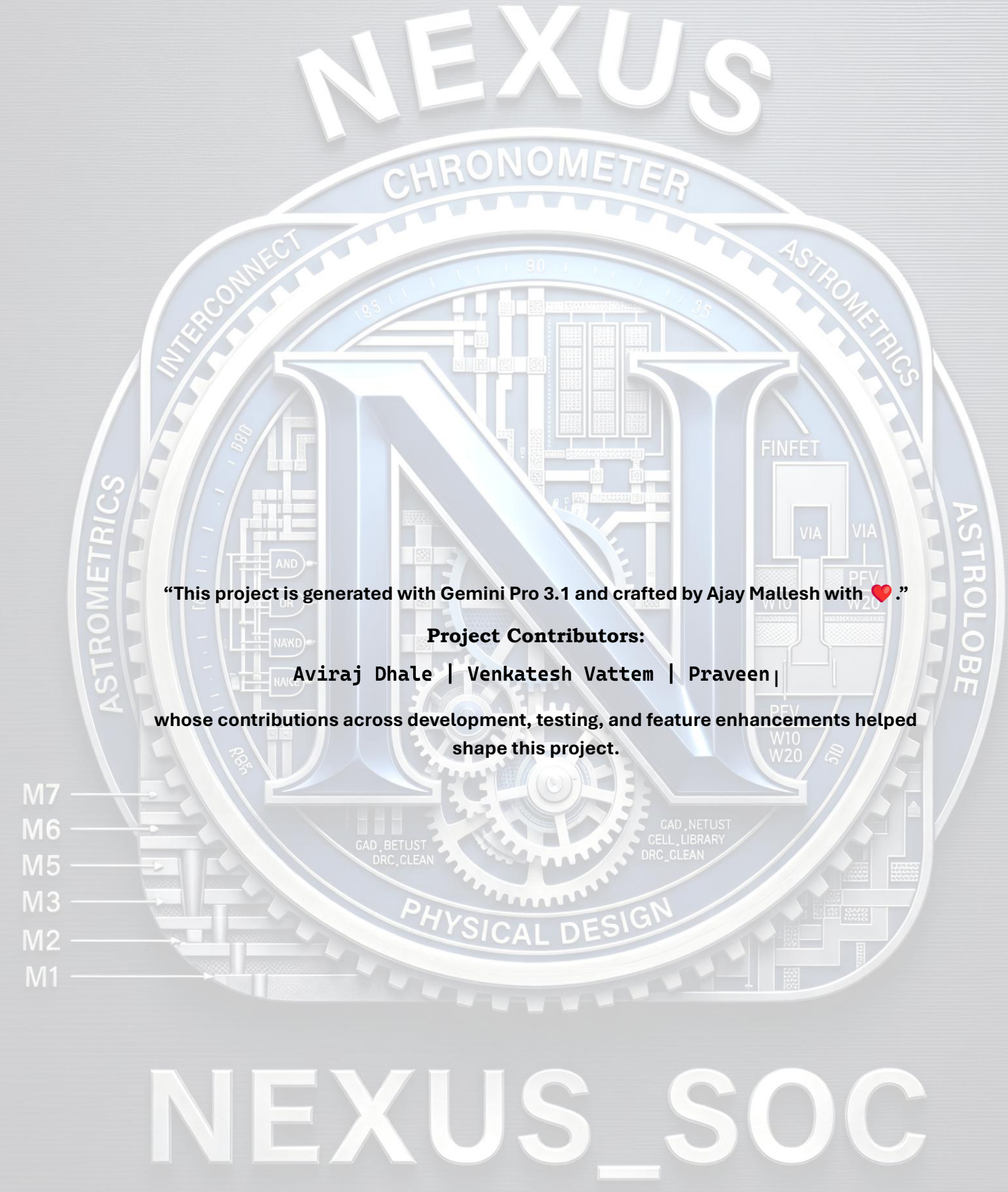
Author: Ajay Mallesh

Reviewer: Gemini Pro 3.1 & Ajay Mallesh

Project Contributors: Aviraj Dhale | Venkatesh Vattem | Praveen

Version: Version 1.0 (2026)

Date: 19/09/2026




**“This project is generated with Gemini Pro 3.1 and crafted by Ajay Mallesh with ❤️.”**

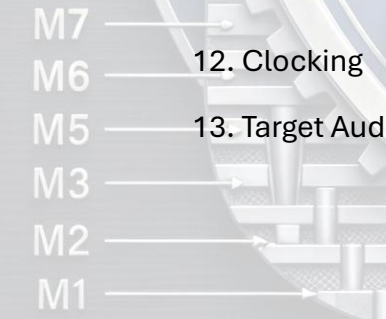
**Project Contributors:**

**`Aviraj Dhale | Venkatesh Vattem | Praveen|`**

**whose contributions across development, testing, and feature enhancements helped shape this project.**

**Technology Focus**

| Parameter | Specification |
|---|---|
| 1. Technology Node | 32nm CMOS (SAED32) |
| 2. Design Type | Multi-Voltage ASIC |
| 3. Core Type | Nexus SoC (nexus_soc_top) |
| 4. Instance Count | ~230K Instances |
| 5. Macro Count & Type | 200 Hard Macros (32 lp_ram_subsystem SRAMs + 100 in High Level + 68 Default) |
| 6. Voltage Domains | 1.05V (VDDH) / 0.85V (VDD) / 0.78V (VDDL) |
| 7. Flow Style | RTL → GDSII |
| 8. Primary Tools | Synopsys (Design Compiler, ICC2, PrimeTime) |
| 9. Timing Methodology | MCMM (Multi-Corner Multi-Mode) |
| 10. Signoff Style | PrimeTime / StarRC |
| 11. Power Strategy | Multi-Domain Mesh PDN (with VDDL + VDDH Guard Bands) |
| 12. Clocking | H-Tree CTS + NDR (Double Spacing) |
| 13. Target Audience | Physical Design Engineers / ASIC Architects |




NEXUS_SOC

## Copyright Notice:

© 2026 Ajay Mallesh. This project is open-source and released for educational and non-commercial community use. You are free to use, modify and distribute this project in accordance with the project license.

Author: Ajay Mallesh

Email: ajaymalavalli912@gmail.com

LinkedIn: AjayMallesh

## Project Contributors:

LinkedIn: Aviraj Dhale

LinkedIn: Venkatesh Vattem

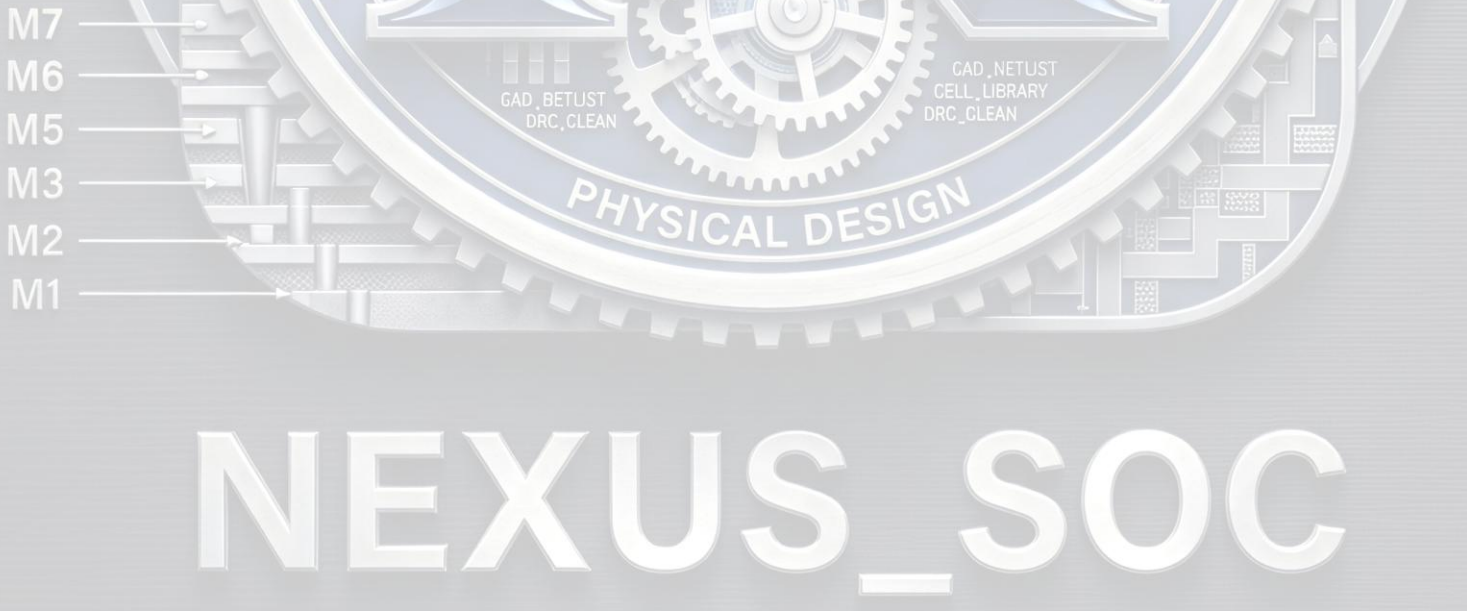
LinkedIn : Praveen Kumar M

## More Physical design Projects:

**MV Macro Engine:** https://ajay-mallesh.github.io/mv_macro_engine_website/

**MV Macro Engine:** https://nexussocwebsite.vercel.app/

**Crypto-top:** https://ajay-mallesh.github.io/crypro_top_website/



**Legal Disclaimer**

The methodologies, TCL examples, UPF constructs, synthesis flows, timing constraints and physical implementation strategies described in this document are intended exclusively for educational and research purposes.

Tool commands and flows may vary depending on:

- tool version,
- process node,
- foundry NDA constraints,
- technology LEF/DEF structure,
- extraction deck version,
- signoff correlation methodology,
- and EDA vendor implementation changes.

The author assumes no responsibility for:

- silicon failure,
- timing closure loss,
- LVS/DRC violations,
- power collapse,
- EM degradation,
- tape-out delay,
- or fabrication loss arising from improper application of the concepts described herein.

NEXUS_SOC

## Contents


Project Introduction & Overview ..... 1

The Challenge of Extreme Voltage Crossings ..... 2

**Chapter 1: Files and Directories (Linux)** ..... 3

**Chapter 2: RTL Implementation** ..... 4

**Chapter 3: Synthesis** ..... 5

**Chapter 4: Floorplan** ..... 8

**Chapter 5: Port Placements** ..... 9

**Chapter 7: Macro Placements** ..... 11

**Chapter 8: Power Delivery Network** ..... 13

**Chapter 9: Congestion checks** ..... 15

**Chapter 10: Placement** ..... 16

**Chapter 11: Clocking Architecture & Specifications** ..... 20

**Chapter 12: Clock Tree Synthesis** ..... 23

**Chapter 13: Routing** ..... 26

**Chapter 14: Signoff (Extract with your own Methods)** ..... 29

**Chapter 15: Sign-off Timing Analysis & ECO Flow** ..... 30

**Chapter 15: Advanced IR Drop** ..... 33

**Chapter 16: Hard Macro Conversion (Block Abstraction)** ..... 35

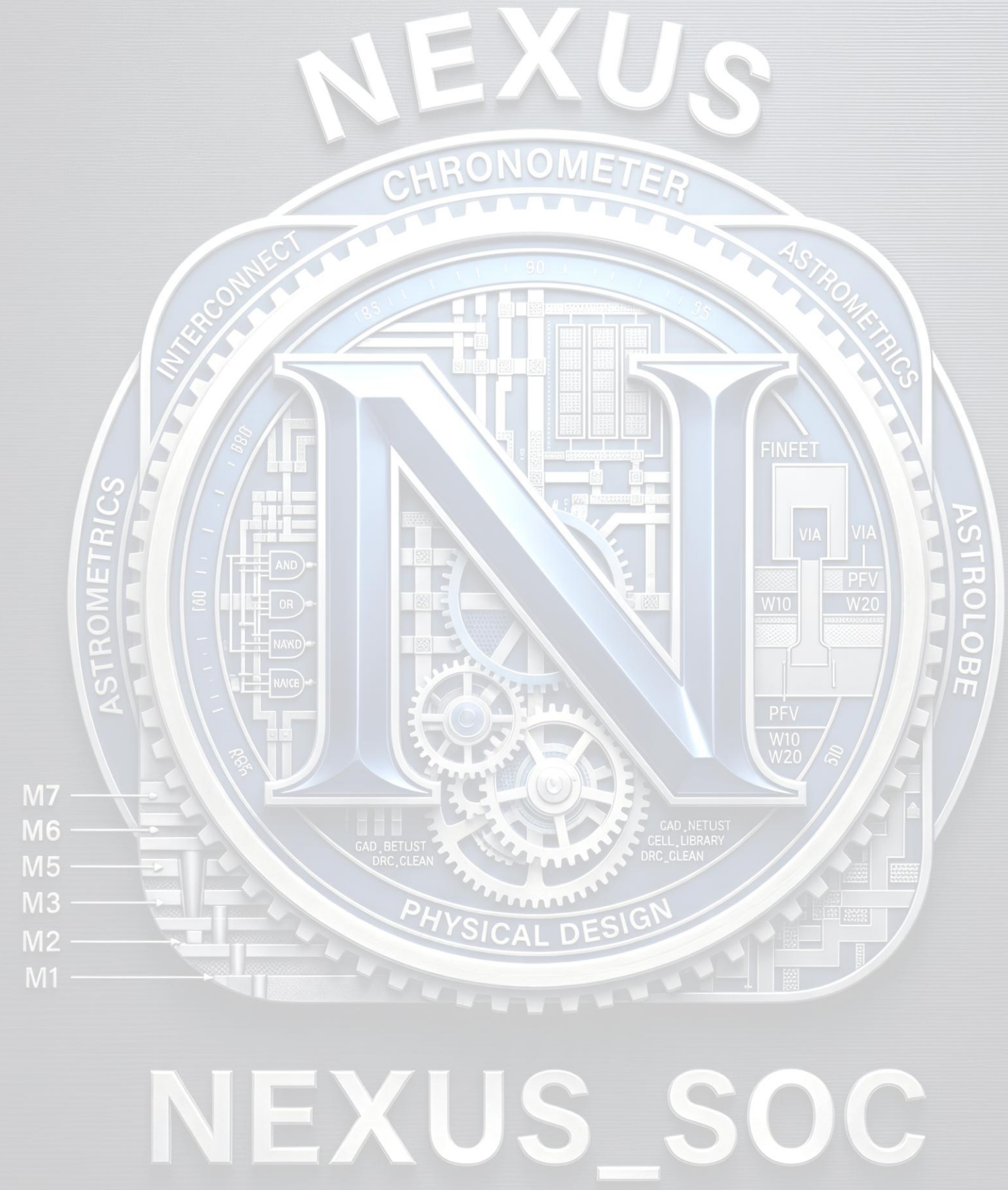
**Chapter 17: Formal Verification (Logic Equivalence Checking)** ..... 38

**Chapter 18: Design for Manufacturing (DFM)** ..... 39

**Chapter 19: The ECO Flow & Final Handoff** ..... 40



NEXUS_SOC


NEXUS
CHRONOMETER
INTERCONNECT
ASTROMETRICS
ASTROMETRICS
ASTROLOBE
FINFET
VIA
VIA
PFV
W10
W20
PFV
W10
W20
AND
OR
NAND
NAICE
GAD_BETUST
DRC,CLEAN
CAD_NETUST
CELL_LIBRARY
DRC_CLEAN
PHYSICAL DESIGN
M7
M6
M5
M3
M2
M1

# NEXUS_SOC

# Project Introduction & Overview

**Multi-Voltage Nexus SoC Architecture** The nexus_soc_top project is a high-performance multi-voltage ASIC design implemented in 32nm CMOS technology. It features a robust architecture comprising 200K standard cells and 200 hard macros, including 32 lp_ram_subsystem SRAMs, 100 hard macros in the High-Level domain, and 68 Default macros.

To optimize for power efficiency and meet strict SoC thermal constraints, the design employs a 3-Multi-Voltage (3MV) architecture based on aggressive voltage scaling. The logic is partitioned into three distinct domains:

- **High-Performance Domain (1.05V / VDDH):** This domain houses the primary computational engine and high-speed logic. It operates at 1.05V to ensure maximum throughput and timing closure, utilizing the saed32rvt_ss1p05v125c.db library.
- **Default Core Domain (0.85V / VDD):** This domain incorporates the standard control logic and default macros. It operates at 0.85V, balancing performance and power, utilizing the saed32rvt_ss0p85v125c.db library.
- **Low-Power Domain (0.78V / VDDL):** This domain incorporates the memory subsystems and low-speed control logic. By operating at 0.78V, the design significantly minimizes both dynamic switching power and static leakage. It utilizes the saed32rvt_ss0p78v125c.db library, optimized for low-voltage operation.
- **Physical Design Challenges** The 3-multi-voltage architecture of the nexus_soc_top introduces several sophisticated backend design requirements:
- **UPF-based Power Intent Integration:** Seamless integration of logical power intent with the physical design database.
- **Level Shifter Management:** Ensuring signal integrity for all crossings between the 1.05V, 0.85V, and 0.78V domains using dedicated ulvl and dlvl level shifters.
- **Macro-Centric Floorplanning:** Strategic placement of 200 hard macros, including the memory subsystems, while maintaining routability and minimizing wire detours.
- **Dedicated Voltage Areas & PDN:** Implementation of a multi-domain mesh Power Delivery Network (PDN) featuring VDDL guard bands for the low-power engine.
- **H-Tree Clock Distribution:** Managing an H-Tree clock topology to minimize skew while handling the multiple voltage-level crossings at the clock spine.
- **MCMM Timing Analysis:** Balancing timing closure across complex multi-corner multi-mode scenarios to ensure robustness across process, voltage, and temperature variations.

**Tools**

- **Synthesis:** Synopsys Design Compiler
- **Physical Design:** Synopsys ICC2
- **Signoff:** Synopsys PrimeTime / StarRC

# The Challenge of Extreme Voltage Crossings

In a standard multi-voltage design, moving a signal between two closely related domains (e.g., 0.85V to 1.05V) is straightforward using standard Low-to-High (L2H) level shifters. However, the nexus_soc_top architecture features an extreme voltage delta between the Low-Power memory domain (VDDL at 0.78V) and the High-Performance computational domain (VDDH at 1.05V). Standard cell libraries often lack a single level shifter capable of reliably bridging such a wide gap without suffering from severe signal degradation, massive delay penalties, or complete logical failure.

**The Cascaded Bridge Mechanism**

To solve this without requiring custom library cells, the design utilizes a cascaded bridge (or "island hopping") methodology. Instead of attempting a direct jump from 0.78V to 1.05V, the architecture leverages the intermediate Default Core Domain (VDD at 0.85V) as a physical and logical stepping stone. The crossing is executed in two distinct, manageable stages:

- **Stage 1 (VDDL to VDD):** The signal exits the 0.78V domain and is routed into the 0.85V domain.
- **Stage 2 (VDD to VDDH):** The signal travels through the default domain until it reaches the high-performance boundary, stepping up to the final 1.05V target.

**Physical Implementation & UPF Strategy** From a physical design perspective, this requires strategic floorplanning and specific UPF (Unified Power Format) rules. To manage the tool's automated insertion and prevent crossing errors, a master voltage bypass is explicitly coded into the UPF. By applying the set_level_shifter -threshold 0.30 constraint to the PD_LP domain, the tool is instructed to bypass mandatory level shifter insertion for any crossing with a voltage gap of 0.30V or less. Because the absolute maximum delta in the SoC is only 0.27V (1.05V - 0.78V), this threshold cleanly allows the cascaded signals to navigate the 0.78V, 0.85V, and 1.05V boundaries without throwing MV-753 and MV-611 DRC errors. The physical placement engine (ICC2) routes the interconnects through the 0.85V physical voltage area (VA) following this structured cascade.

**Key Advantages for the Nexus SoC**

- **Library Compatibility:** Completely eliminates the need for specialized, extreme-range level shifters, allowing the design to rely entirely on the standard saed32rvt DB files.
- **Tool Error Resolution:** The 0.30V threshold bypass elegantly resolves multi-voltage crossing DRCs while fully preserving the cascaded 3-domain architecture.
- **Timing Reliability:** Breaking the transition into smaller, managed jumps provides far more predictable and stable timing arcs for the PrimeTime signoff engine.
- **Signal Integrity:** Prevents the slew degradation and noise susceptibility that occurs when driving a signal across a massive voltage threshold

## Chapter 1: Files and Directories (Linux)

### 1.1. Directory creations

- **Command:** mkdir nexus_soc
- cd nexus_soc/

### 1.2. Files creations

- **Command:** gvim <file_name>
- cat <file_name>
- nano <file_name>

### 1.3. Automated Script

- source setup_env.tcl

*** This script will create all the required directories

NEXUS
CHRONOMETER
ASTROMETRICS
ASTROLOBE
INTERCONNECT
FINFET
VIA
PFV
W10
W20
AND
OR
NAND
NAICE
M7
M6
M5
M3
M2
M1
GAD_BETUST
DRC,CLEAN
CAD_NETUST
CELL_LIBRARY
DRC_CLEAN
PHYSICAL DESIGN
NEXUS_SOC

## Chapter 2: RTL Implementation

### 2.1 RTL File Implementation Logic

For a multi-voltage design, the RTL logic must be cleanly partitioned into distinct hierarchical blocks based on their target voltage. In the nexus_soc_top design, the architecture is split into three main sections to support the 3-Multi-Voltage (3MV) implementation:

- **1. The High-Performance Domain (1.05V / VDDH):** Runs on the highest 1.05V supply. This domain handles the primary computational engine and high-speed logic interfaces.
- **2. The Default Core Domain (0.85V / VDD):** Runs on the intermediate 0.85V supply. This block incorporates the standard control logic and default macros, while also acting as a critical physical and logical stepping stone for signals crossing between extreme voltages.
- **3. The Low-Power Domain (0.78V / VDDL):** Runs on the scaled 0.78V supply. This domain incorporates the memory subsystems—including the 32 lp_ram_subsystem SRAMs and heavy, latency-tolerant computation to minimize leakage and dynamic power.

The top-level Verilog (nexus_soc_top ) wrapper acts as the integration layer that structurally wires these three distinct blocks together.

### 2.2 Level Shifters Declarations & Cascaded Bridge

- When a signal travels from a lower voltage domain back to a higher voltage domain, a Low-to-High Level Shifter is required. Without it, the lower voltage "High" signal won't fully turn off the higher domain's PMOS transistors, causing a short circuit (crowbar current). In the nexus_soc_top architecture, crossing directly from the 0.78V (VDDL) domain to the 1.05V (VDDH) domain presents an extreme voltage delta that standard standard cells cannot reliably bridge in a single step.
- **How they are declared:** In modern physical design, we do not hardcode level shifters or cascaded bridge logic directly into the RTL. The RTL simply connects the interconnects normally between the hierarchical domains.
- **UPF Integration:** Instead, level shifters are declared and automatically managed using the UPF (Unified Power Format) file during synthesis. To handle the extreme 0.78V to 1.05V crossing without tool errors, the UPF utilizes a specific voltage difference threshold bypass (-threshold 0.30). This legally instructs the synthesis and physical design engines to bypass direct level shifter insertion for large voltage gaps, forcing the signals to route seamlessly through the 0.85V default domain via a two-stage cascaded bridge.

## Chapter 3: Synthesis

### 3.1 What is Synthesis?

Synthesis is the process of translating human-readable RTL code (like Verilog) into a technology-mapped gate-level netlist. The tool reads the behavioral logic and converts it into physical logic gates (AND, OR, Flip-Flops) available in the target 32nm standard cell library, while trying to meet timing and power requirements.

#### 3.1.1 Types of Synthesis

- **Logical Synthesis:** Traditional synthesis that assumes zero wire delays. It only looks at the logic gates.
- **Physical Synthesis (Topographical):** Advanced synthesis that predicts physical layout (placement and wire lengths) to give much more accurate timing and power estimates before the actual Physical Design phase.

### 3.2 Synthesis used in this design

For the nexus_soc_top, we use Physical (Topographical) Synthesis using Synopsys Design Compiler (DC). Because this is a 3-Multi-Voltage (3MV) architecture, the synthesis tool must be Multi-Voltage Aware. It reads the power intent to manage the 1.05V (VDDH), 0.85V (VDD), and 0.78V (VDDL) domains, automatically inserting standard level shifters and respecting the cascaded bridge thresholds for extreme voltage crossings.

### 3.3 Files / Inputs required for Synthesis

To run synthesis, the tool needs five critical inputs:

#### 3.3.1 RTL

The verified Verilog files (e.g., pci_tx_rx.v, cascade_bridge.v, lp_ram_subsystem.v, nexus_soc_top.v) containing the design logic.

#### 3.3.2 .lib (Logic Libraries)

The logic libraries provided by the foundry. Because we have three distinct domains, we must load the standard cell DBs for all operating voltages (saed32rvt_ss1p05v125c.db, saed32rvt_ss0p85v125c.db, and saed32rvt_ss0p78v125c.db), usually at worst-case timing corners.

#### 3.3.3 UPF

The IEEE 1801 Unified Power Format file (nexus_power_intent.upf). This dictates the VDDH, VDD, and VDDL domains, the supply network, and the specific 0.30V threshold bypass rules for the cascaded level shifters.

### 3.3.4 SDC

Synopsys Design Constraints (nexus_constraints.sdc). This file defines the clock frequencies, input/output delays, 30% timing derates, and false paths.

### 3.3.5 Scanchains

Design for Testability (DFT) setup files. These tell the tool how to stitch the flip-flops together into long "scan chains" so the chip can be tested for manufacturing defects after fabrication.

### 3.4 Synthesis script file - ./scripts/dc_compile.tcl

A simplified look at the main commands inside the synthesis script:

Tcl (Simplified Example) / Not to execute this alone

```
# (Simplified Example) / Not to execute this alone
# 1. Read the design
analyze -format sverilog {./inputs/pci_tx_rx.v
./inputs/cascade_bridge.v ./inputs/lp_ram_subsystem.v
./inputs/nexus_soc_top.v}
elaborate nexus_soc_top

# 2. Load constraints and power intent
source ./inputs/nexus_constraints.sdc
load_upf ./inputs/nexus_power_intent.upf

# 3. Stitch scan chains for DFT
insert_dft

# 4. Compile the design into 32nm gates
compile_ultra -scan -timing_high_effort_script
```

### 3.5 Generated output files

Once compile_ultra finishes, it generates the following deliverables for the Physical Design (PnR) team:

### 3.5.1 Golden Netlist

".v" file containing only 32nm standard cell instances and their wire connections. All abstract RTL is gone.

### 3.5.2 UPF

An updated .upf file. The tool updates the original nexus_power_intent.upf to include the exact names and locations of the level shifters it inserted.

### 3.5.3 Scandef

A .scandef file containing the scan chain trace. This ensures the PnR tool (ICC2) knows how the flip-flops are chained together and can optimize their placement.

### 3.5.6 SDC (not recommended for Physical Design)

The tool generates a forward-annotated SDC file. Note: It is generally not recommended to use this generated SDC for physical design. It often contains cluttered, auto-generated tool variables. It is best practice to pass the clean, original nexus_constraints.sdc to the PnR team.

## 3.6 Reports

Before handing the files off to PnR, these logs must be reviewed:

### 3.6.1 report_timing

Checks the critical paths. Setup timing should be met (or very close to met) here. Hold timing is generally ignored at this stage because the clock tree doesn't exist yet.

### 3.6.2 report_area

Checks the total cell count and silicon footprint. We verify that the design is roughly hitting our target of ~200K instances and 200 hard macros.

### 3.6.3 report_qor

The Quality of Results report. Provides a quick summary of the worst negative slack (WNS), total negative slack (TNS), cell count and leakage power.

## 3.7 Sanity checks

To sign off on synthesis, two crucial checks must pass clean:

### 3.7.1 check_design

Scans the netlist for logical errors. You must ensure there are zero floating inputs and zero multi-driven nets (where two outputs try to drive the same wire).

### 3.7.2 check_mv_design

Scans the multi-voltage architecture. This ensures that every extreme voltage crossing from the 0.78V VDDL domain to the 1.05V VDDH domain successfully utilized the 0.85V VDD cascaded bridge, and that the 0.30V UPF bypass prevented any illegal direct connections.

## 3.8 Automated Script for synthesis

- source ./scripts/synthesis_nexus.tcl

*** This will do synthesis and generate the output files. Change Path accordingly ***

# Chapter 4: Floorplan

## 4.1 Creating NDM

Before physical design can begin, the tool needs a unified database containing both logical timing and physical layout information. In Synopsys ICC2, this is the New Data Model (NDM). We create the NDM by merging the synthesized .v netlist, the .db timing libraries, and the physical .lef/FRAM files (including saed32_lp9m_tech.ndm and saed32_sram_lp.ndm) for the 1.05V (VDDH), 0.85V (VDD), and 0.78V (VDDL) domains.

## 4.2 Initialize Floorplan

Floorplanning establishes the physical silicon canvas for the 200K instances and 200 hard macros. This phase is executed using the automated script located at `./scripts/floorplan/Floorplan_script.tcl`.

### 4.2.1 Read Netlist

The tool loads the structural netlist (nexus_soc_top_netlist_sfo.v) generated by the synthesis team, mapping the logical instances to physical 32nm standard cell shapes.

### 4.2.2 Create Core and Die Area

We define the overall chip dimensions. The Die Area includes the outer I/O pad ring, while the Core Area is the internal region where the standard cells and macros are placed. For the nexus_soc_top, we use a rectilinear shape with a 1:1 aspect ratio, a 5-micron core boundary offset, and a strict utilization target of 55% (0.55). This precise 55% utilization ensures enough blank space is left for the 200 hard macros (including the 32 lp_ram_subsystem SRAMs), routing wires, and inserting clock buffers later.

M7
M6
M5
M3
M2
M1

**Example:**

```
initialize_floorplan -shape R -side_ratio {1 1} -core_utilization 0.55 -core_offset 5 -site_def unit -use_site_row
```

## 4.3 Sanity checks

### 4.3.1 check_floorplan

Validates that the core area is legally defined, boundaries are closed, and there are no overlapping hard macros that violate placement rules.

## Chapter 5: Port Placements

### 5.1 Block Pin Constraints - Placing ports

In a hierarchical design like the nexus_soc_top, the external input/output signals must enter and exit the block through physical pins on the core boundary. Pin constraints define exactly which edge (top, bottom, left, right) and which metal layers the ports should use, optimizing the dataflow into the high-speed computational engine (VDDH / 1.05V) and the low-power subsystems (VDDL / 0.78V).

- For this project, you have the flexibility to define your own custom port placements based on your specific top-level integration requirements. Alternatively, you can execute the pre-defined port locations using the provided reference files:
- **Port Constraints:** ./docs/port locations/ports_locations.tcl
- **Execution Script:** ./scripts/ macros_and_ports_placement_script/ ports_placement_individual_script.tcl

*Note: High-speed clock and critical data ports are restricted to upper metal layers (e.g., M5/M6) to minimize RC delay entering the core.*

### 5.2 Sanity checks

#### 5.2.1 check_pin_placement

Confirms that all pins have been successfully dropped onto the core boundary and align properly with the 32nm routing grid.

#### 5.2.2 check_pre_pin_placement

A preventative check run before physical placement. It verifies that your pin constraint file is syntactically valid and that you aren't trying to place pins on an illegal metal layer or a highly congested edge (such as regions blocked by the 200 hard macros).

## Chapter 6: Multi-voltage Design

### 6.1 UPF

We read the .upf file outputted by Synthesis into ICC2

(**load_upf ./outputs/nexus_soc_final /nexus_soc_top.upf**). This ensures the physical tool understands exactly which cells belong to the three distinct power domains: the high-performance domain (1.05V), the standard domain (0.85V), and the low-power domain (0.78V). We also manually set the voltages (**set_voltage**) via terminal / script commands, ensuring this command data is strictly kept out of the UPF file—to prevent UPF checker errors (UPF-057) before proceeding.

### 6.2 create voltage area

We physically draw the PD_HIGH and PD_LP regions on the floorplan canvas, with the remainder of the core acting as the DEFAULT_VA. This optimized boundary creation is executed using the script located at

**./scripts/voltage_area/voltage_area_nexus_soc.tcl**

The tool will strictly place the standard cells inside their respectively designated boundaries. Critically, all 32 lp_ram_subsystem SRAM hard macros are physically constrained to reside entirely within the PD_LP (0.78V) domain, completely separated from the 1.05V and 0.85V logic.

### 6.3 commit UPF

M7
M6
M5
M3
M2
M1

This command tells ICC2 to finalize the power intent. It maps the logical UPF definitions to the three physical voltage areas we just drew and creates the actual power nets (such as vdd and vddl and vddh) in the database.

### 6.4 Sanity checks

#### 6.4.1 check_mv_design

Validates that the physical floorplan correctly supports the 3-multi-voltage intent without illegal overlaps between DEFAULT_VA, PD_HIGH, and PD_LP.

#### 6.4.2 report_mv_cells

Generates a list of all level shifters and isolation cells, confirming they were successfully carried over from synthesis.

#### 6.4.3 report_mv_path -level_shifter -all_path

Traces every wire crossing between the 0.78V, 0.85V, and 1.05V areas to guarantee the correct level shifter or bypass bridge physically sits between them, preventing crowbar currents.

# Chapter 7: Macro Placements

## 7.1 Macro Placement Strategy

With the floorplan initialized, ports placed, and the multi-voltage areas (UPF) successfully defined, the next critical step is placing the hard macros. Macro placement dictates the routability and timing success of the entire physical design. Macros generally act as physical blockages and should be placed toward the core boundaries to avoid fracturing the standard cell placement area.

***Note**: It is not strictly necessary to use the predefined macro placements provided in this reference kit. You have the flexibility to place the macros according to your own custom dataflow requirements, logical flylines, and top-level SoC constraints.*

## 7.2 Execution Scripts and Location Files

- If you choose to use the verified reference floorplan to guarantee routability for the nexus_soc_top, the precise X/Y coordinates and orientations for all 200 macros (32 lp_ram_subsystem SRAMs and 168 performance/default macros) are defined and executed using the following files:
- **Macro Locations File:** ./docs/macro_locations/macro_locations.tcl
- **Execution Script:** ./scripts/macros_and_ports_placement_script/macro_placement_script.tcl

## 7.3 Domain-Specific Macro Constraints

Because this is a multi-voltage design, macro placement must strictly adhere to the physical power domains established in the previous chapter:

- **Low-Power SRAMs:** The 32 lp_ram_subsystem macros must be placed entirely inside the PD_LP (0.78V) voltage area boundary so they properly connect to the vddl power grid.
- **High-Performance & Default Macros:** The remaining 168 macros are placed across the PD_HIGH (1.05V) and DEFAULT_VA (0.85V) domains according to their specific performance requirements.
- **Halos and Blockages:** Placement halos (keepout margins) are automatically applied around all macros by the script to prevent standard cells from packing too closely to the macro pins, which reduces local routing congestion.

### 7.4 Sanity checks

#### 7.4.1 check_legality -macros_only

Validates that there are zero overlapping hard macros, that all macros are legally snapped to the 32nm manufacturing grid, and that their physical orientations are permitted by the foundry rules.

#### 7.4.2 check_mv_design

Run immediately after macro placement to ensure that none of the 32 lp_ram_subsystem macros have accidentally breached the boundaries of the **PD_LP** voltage area, which would cause severe multi-voltage power domain violations.

## Chapter 8: Power Delivery Network

### 8.1 Create Tap and bound cells

Before laying metal, we place specific physical-only standard cells:

- **Tap Cells:** Placed at regular intervals to tie the silicon substrate to ground and n-wells to power, preventing latch-up.
- **Boundary Cells (Endcaps):** Placed at the end of every standard cell row to protect the delicate logic gates from manufacturing damage.

### 8.2 Sanity checks

#### 8.2.1 check_boundary_cells

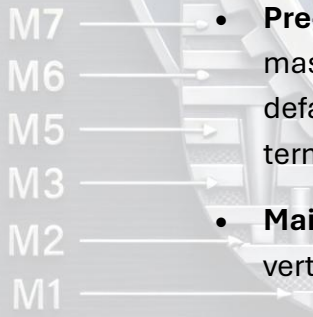
Ensures all rows are properly capped and that the maximum allowed distance between tap cells is not violated.

### 8.3 creating PG nets

We construct the power grids (VDD, VDDH, VDDL, and VSS) to distribute current evenly across the three voltage domains and prevent IR-drop. The standalone execution script for this stage

**./scripts/powerplan/powerplan_nexus_soc.tcl**

The implementation follows a highly explicit routing topology:




- **Pre-Power Netlist ECO:** We physically disconnect tied-off signal ports (e.g., master_sig_out) from VSS nets. This surgical ECO prevents the tool from treating default-value logical pins as physical ground pins, completely eliminating terminal overlap DRCs without requiring a full RTL re-synthesis.
- **Main Power Mesh:** The coarse upper grid utilizes M7 horizontally and M8 vertically, which drops down to a fine M2 triple-track lower mesh.
- **Macro Rings:** Dedicated macro rings are constructed using M5 horizontally and M6 vertically, encompassing the default, high-performance (PD_HIGH), and low-power (PD_LP) macros.
- **Macro Pin Connections:** Macro pins are tied directly to the M5/M6 rings using a scattered pin connection pattern.
- **Standard Cell Rails:** Standard cell rails are successfully routed and dropped into their respective voltage areas, ensuring PD_LP receives VDDL, PD_HIGH receives VDDH, and the core receives VDD.

### 8.4 Sanity checks

#### 8.4.1 check_pg_connectivity -check_std_cell_pins none

Verifies the macro-level power grid is fully connected to the supply ports without getting bogged down by checking millions of individual standard cell pins.

#### 8.4.2 check_pg_missing_vias

Ensures that where vertical and horizontal power straps cross, a physical via successfully fuses them together.

#### 8.4.3 check_pg_drc -do_not_check_in_hier_shapes_blocks

Checks the thick power metal routes for spacing and width Design Rule Check (DRC) violations, ensuring the foundry can manufacture the grid without shorts.




NEXUS_SOC

## Chapter 9: Congestion checks

Before committing to the heavy, time-consuming optimization phases, we must perform a "dry run" to ensure our floorplan, the 200 macro placements, and the 3-multi-voltage areas won't cause a massive traffic jam of routing wires later on.

### 9.1 create_placement -floorplan

Instead of placing cells meticulously for timing, this command performs a fast, coarse placement of the ~200K standard cells. The -floorplan flag tells the tool to ignore heavy timing optimizations and simply drop the standard cells into the core (strictly respecting the 1.05V PD_HIGH, 0.85V DEFAULT_VA, and 0.78V PD_LP voltage areas) to see how crowded the silicon canvas gets.

### 9.2 congestion reports

Once the cells are temporarily placed, we need to analyze if there is enough physical space (metal tracks) for all the logical connections to be wired together.

#### 9.2.1 report_congestion -rerun_global_router

This command is critical. It triggers the global routing engine to do a quick, rough estimate of how the wires will be drawn.

- The -rerun_global_router switch forces the tool to calculate fresh routing data based exactly on the fast placement we just did.
- We review the log to identify "hotspots." If the tool reports high overflow (meaning more wires need to pass through a channel than there are physical metal tracks available, especially in the narrow channels between the 32 lp_ram_subsystem macros), the floorplan is unroutable. You must stop, adjust your macros or pin constraints, and try again.

### 9.3 reset_placement

Since create_placement -floorplan was only a temporary test to check for congestion hotspots, we do not want to keep it. Once we confirm the floorplan is highly routable, we execute reset_placement. This rips up the quick standard cell placement, leaving behind our clean, verified floorplan and power grids ready for formal placement.

## Chapter 10: Placement

Placement is the phase where the tool formally assigns a physical X,Y coordinate to all ~200K standard cells. The goal is to optimize logic for setup timing, minimize total wire length, and strictly obey the 1.05V (PD_HIGH), 0.85V (DEFAULT_VA), and 0.78V (PD_LP) voltage boundaries, all without creating routability issues.

### 10.1 Pre-requisite: Scandef Setup

Before executing the placement engine, we must load the scan chain definitions generated during synthesis. This ensures the tool knows how the flip-flops are chained together and can place them optimally to avoid routing congestion later.

- **Command:**

  **read_def ./outputs/nexus_soc_final/nexus_soc_top.scandef**

### 10.2 Source mcmm_setup file - modes & scenarios

Before placement optimization begins, Multi-Corner Multi-Mode (MCMM) constraints must be loaded. The placement engine must "see" all operational scenarios simultaneously.

- **Command:**

  **source ./inputs/ mcmm_files/mcmm_NEXUS_SOC.tcl**

  *(Note: Make sure to verify all inner paths, such as mcmm_corners_modes.tcl, to ensure the corner and mode definitions are accurately pointing to your .lib files).*

M7
M6
M5
M3
M2
M1

For this nexus_soc_top design, it must balance the timing paths across different Process, Voltage, and Temperature (PVT) corners (e.g., worst-case timing across the 1.05V, 0.85V, and 0.78V domains at 125°C, versus best-case at -40°C). Fixing a path in one corner without MCMM loaded can easily break it in another. Additionally, specific timing exceptions are applied at this stage, such as a hold multi-cycle path (value -1) crossing from the clk_ft_0 domain to the clk_master domain.

### 10.3 Timing Exceptions: Multi-Cycle Paths (MCP)

While MCMM setup handles global timing scenarios, specific logic paths often require manual timing exceptions to prevent the placement engine from over-optimizing. A Multi-Cycle Path (MCP) instructs the tool that data traveling between two specific endpoints is intentionally designed to take more than one clock cycle to propagate.

In the placement_nexus_script.tcl, we explicitly apply a hold timing exception between two clock domains:

**set_multicycle_path -hold -1 -from [get_clocks clk_ft_0] -to [get_clocks clk_master]**

- **The Setup/Hold Relationship:** By default, if a setup path is relaxed (e.g., given 2 clock cycles instead of 1), the tool automatically shifts the *hold* check forward as well. This causes the tool to falsely believe the data might arrive *too early* on the new cycle, prompting it to insert massive chains of artificial delay buffers.
- **The -hold -1 Correction:** This command explicitly shifts the hold check backward by one clock cycle, anchoring it back to the original launch edge.
- **Physical Impact:** By preventing the place_opt engine from needlessly dumping hundreds of hold-fixing buffers into the data paths crossing between clk_ft_0 and clk_master, we directly save die area, reduce local routing congestion, and eliminate the leakage power those extra buffers would have consumed.

### 10.4 Source Placement Script

Instead of running commands manually, the placement engine is executed via this master script, which performs global placement, legalizes the cells onto the standard cell rows, and performs High-Effort logic optimization to fix setup timing violations using the place_opt command.

Crucially, the script establishes physical boundaries before routing starts by setting the maximum coarse density to 55% to reserve area for future clock buffers, restricting logic cell fanout to a maximum of 40 endpoints, and confining placement routing estimates strictly to layers M2 through M6. It also enables advanced legalization algorithms for better spatial repair.

- **Command:**

source ./scripts/Placement_script/placement_nexus_script.tcl

### 10.5 Sanity checks

#### 10.5.1 check_legality

Validates that every single standard cell is placed legally. This means:

- Cells are aligned to the manufacturing site rows.
- Zero cells are overlapping.
- Zero 0.78V logic cells have accidentally leaked out of the PD_LP voltage area.
- Zero 1.05V logic cells have accidentally leaked out of the PD_HIGH voltage area.

### 10.6 Reports

After placement, timing violations are expected, but physical Design Rule Checks (DRC) like transition and capacitance must be heavily scrutinized.

#### 10.6.1 report_constraints -all_violators -max_transition -significant_digits 6

Identifies nets where the signal takes too long to toggle from 0 to 1 or 1 to 0 (slew rate). Slow transitions cause sluggish performance and massive short-circuit power dissipation.

#### 10.6.2 report_constraints -all_violators -max_capacitance -significant_digits 6

Identifies cells trying to drive a load capacitance that exceeds the maximum limit characterized in the .lib files. This happens when a net is too long or a cell has too many fanouts.

### 10.7 Fixes for max trans

If the tool's auto-optimization leaves max transition violations, they must be resolved. Transition degradation is mostly caused by excessive RC wire delay.

#### 10.7.1 Upsize cells (Not Recommended) - script: ./script/upsize_driver.tcl

Swaps the driving cell for a larger footprint cell with stronger drive strength. Not recommended manually because blindly upsizing steals area, increases power consumption, and often creates congestion hotspots in dense areas.

#### 10.7.2 Insert Buffers (Not Recommended) - script: ./script/insert_buffer.tcl

Cuts a long, resistive wire in half by inserting a buffer to restore the signal slew. Not recommended manually at this stage because the clock tree doesn't exist yet; manual buffers can severely complicate CTS (Clock Tree Synthesis) later.

#### 10.7.3 Magnet placement

A physical constraint technique. You define a "magnet" (often a macro or a critical level shifter) and the tool pulls all logically connected standard cells physically closer to it. Shorter wires mean less resistance, naturally fixing the transition violation.

#### 10.7.4 Create_bounds

Forces a specific group of critical cells into a tightly restricted physical box. By caging the cells together, you prevent the placement engine from spreading them out across the die during optimization, controlling wire length.

#### 10.7.5 Refine_placement

An incremental command run after applying bounds or magnets. It gently shifts surrounding cells to legalize the new constraints without destroying the overall placement optimization you already achieved.

#### 10.7.6 Path grouping

Assigns a higher "weight" or priority to specific failing paths (for example, the data bus crossing from the 0.78V domain to the 0.85V domain via the cascaded bridge). This forces the optimization engine to spend more computational effort fixing transitions on those specific nets at the expense of non-critical paths.

### 10.8 Fixes for max cap

Max capacitance violations are primarily driven by fanout (one cell trying to drive too many receivers).

#### 10.8.1 Upsize cells (Not Recommended) - script: ./script/upsize_driver.tcl

Using a stronger gate to handle the large capacitive load. As before, this is a brute-force approach that degrades area and power metrics.

#### 10.8.2 Insert Buffers (Not Recommended) - script: ./script/insert_buffer.tcl

Isolating the heavy load from the original driver by buffering the net.

#### 10.8.3 Split load

Instead of one cell driving 40 endpoints, you insert parallel buffers. Buffer A drives 20 endpoints and Buffer B drives the other 20. This halves the capacitance seen by the drivers, fixing the DRC violation efficiently.

#### 10.8.4 Cloning

A highly effective logic transformation. You instruct the tool to create an exact duplicate (clone) of the struggling logic cell. The tool then routes the inputs to both cells and distributes half the output fanout to the original cell and half to the clone. This perfectly resolves max cap issues on critical control signals without adding unnecessary buffer delays.

NEXUS_SOC

## Chapter 11: Clocking Architecture & Specifications

The Multi-Voltage Nexus SOC timing constraints define 11 propagated (physical) clocks and 1 virtual clock across three distinct voltage domains.

### 11.1 SoC Clock Definitions

| Clock Name | Time Period (ns) | Frequency (MHz) | Domain |
|---|---|---|---|
| clk_master | 2.5 | 400 | 0.85V |
| clk_sdram_0 | 3.0 | 333.33 | 0.85V |
| clk_sdram_1 | 3.0 | 333.33 | 0.85V |
| clk_sdram_2 | 3.0 | 333.33 | 0.85V |
| clk_sdram_3 | 3.0 | 333.33 | 0.85V |
| clk_ft_0 | 5.0 | 200 | 0.85V |
| clk_ft_1 | 5.0 | 200 | 0.85V |
| clk_h_tx | 1.5 | 666.67 | 1.05V |
| clk_h_rx | 1.5 | 666.67 | 1.05V |
| clk_lp_tx | 12.0 | 83.33 | 0.78V |
| clk_lp_rx | 12.0 | 83.33 | 0.78V |
| clk_virtual | 3.0 | 333.33 | Ideal/Reference |

**Note:**

- **Default Domain – 0.85V**
- **High Power Domain – 1.05V**
- **Low Power Domain – 0.78V**
- For Frequency calculation, **we used 1000/Tp to keep in MHz**

The timing constraints (SDC) define 11 propagated (physical) clocks and 1 virtual clock.

### 11.2 Default Power Domain Clocks (VDD - 0.85V)

This is the primary operating domain of the SoC, handling core logic, memory interfaces, and system monitoring.

- **clk_master (Period: 2.5ns / 400 MHz):** The main system clock driving the central processing logic and interconnects.

- **clk_sdram_0 to clk_sdram_3 (Period: 3.0ns / 333 MHz):** Dedicated quad-channel memory clocks. Operating slightly slower than the master clock ensures stable read/write operations to external SDRAM without violating setup timing across long bus routes.
- **clk_ft_0 & clk_ft_1 (Period: 5.0ns / 200 MHz):** Fault-tolerant (FT) monitoring clocks. These run at a heavily relaxed frequency to ensure maximum reliability when tracking system health.

**11.2 High Power Domain Clocks (VDDH - 1.05V)**

This domain is dedicated to high-bandwidth, latency-critical interfaces. The higher voltage allows the silicon to switch faster without signal degradation.

- **clk_h_tx & clk_h_rx (Period: 1.5ns / 666 MHz):** The transmit and receive clocks for the high-speed data links. These are the most timing-critical physical clocks in the SoC and dictate the maximum throughput of the chip.

**11.3 Low Power Domain Clocks (VDDL - 0.78V)**

This domain is reserved for background tasks, low-speed peripherals, and power-saving modes. The lowered voltage significantly reduces active and leakage power.

- **clk_lp_tx & clk_lp_rx (Period: 12.0ns / 83.3 MHz):** Low-speed serial transmit and receive clocks. While the timing constraints here are highly relaxed, routing these clocks requires careful buffer selection to prevent massive insertion delay degradation caused by the 0.78V supply.

**11.4 Virtual Clock Constraints**

- **clk_virtual (Period: 3.0ns / 333 MHz):** This clock does not physically exist inside the SoC. Its sole purpose is to serve as an ideal timing reference for the Input/Output boundaries. By defining clk_virtual, the tool can accurately constrain set_input_delay for sys_data_in* and set_output_delay for master_sig_out*, ensuring the physical silicon can safely interface with external board-level components.

### 11.5 Asynchronous Clock Grouping

To prevent the timing engine from wasting computational effort trying to fix non-existent or impossible timing paths, the set_clock_groups -asynchronous command logically isolates the clocks into four distinct groups corresponding to their domains (Default, High, Low, and Virtual). Paths crossing between these groups are treated as asynchronous, meaning standard setup/hold checks are ignored in favor of dedicated synchronizer logic.

### 11.6 Multi-Cycle Path (MCP) Integration

While most clocks are grouped asynchronously, specific architectural exceptions exist within the same voltage domain. The SDC deliberately establishes a Multi-Cycle Path (MCP) between clk_ft_0 (200 MHz) and clk_master (400 MHz). Because the fault-tolerant clock runs at exactly half the speed of the master clock, the design safely utilizes a 2-cycle setup allowance (set_multicycle_path -setup 2) when moving diagnostic data from the fast master domain into the slower monitoring domain. We will explore the deep mechanics of these timing exceptions and how the tool resolves them in a subsequent chapter.

NEXUS
NEXUS_SOC

## Chapter 12: Clock Tree Synthesis

Up until this phase, the clock was "ideal" (zero delay, infinite drive strength). Clock Tree Synthesis (CTS) physically builds the clock distribution network, inserting buffers and routing wires to deliver the clock signal to all flip-flops simultaneously.

In a multi-voltage design like the Multi-Voltage Nexus SOC, clock domains crossing between different voltage levels must be heavily scrutinized to minimize insertion delay differences caused by the voltage drop.

### 12.1 Type of CTS used - H-Tree

For this design, we utilize an H-Tree Clock Routing topology via ICC2's multi-source clock subtree options. Instead of an unstructured tree, the clock is distributed symmetrically across the core through a highly balanced, repeating "H" pattern. This structural approach minimizes On-Chip Variation (OCV) and avoids top-level power planning metals by explicitly restricting root and internal clock routes to intermediate layers.

### 12.2 source ./scripts/CTS_H_TREE/cts_H_tree_nexus.tcl

This master script executes the complete clock building process for the standard cells. The execution flow is strictly controlled through the following stages:

- **Pre-CTS Cleanup:** The script explicitly creates VDD, VDDH, and VDDL power nets if missing, removes dont_touch restrictions on clock nets and level shifters, and strips ideal_network flags so the CTS engine sees the true physical load.
- **Lib Cell Purpose & Reference Setup:** By default, tool cell assumptions are cleared. We explicitly source the allowed library references and promote specific standard high-drive buffers, inverters, registers, and level shifters (NBUFF*LVT, NBUFF*RVT, INVX*_LVT, INVX*_RVT, *DFF*, LSDNSSX1*) for clock tree usage.
- **NDR Setup & Layer Constraints:** Applies double-width and double-spacing rules (iccrm_clock_double_spacing) to the clock nets.
    - **Sink Routing:** Restricted to M4 and M5.
    - **Root & Internal Routing:** Restricted to M5 and M6. This intentionally preserves higher metal layers entirely for the robust multi-voltage power mesh.
- **Clock Constraints & MCMM:** Max transition is clamped at 0.15. Target skews are strictly corner-specific: 0.05ns for slow-slow (ss_125c) and a tighter 0.02ns for fast-fast (ff_m40c). Clock uncertainties are initialized at 0.1 for setup and 0.05 for hold.

- **CRPR Validation:** Clock Reconvergence Pessimism Removal (CRPR) is enabled (time.remove_clock_reconvergence_pessimism) to ensure the timing engine doesn't artificially penalize common clock path segments.
- **Level Shifter Insertion & Subtree Synthesis:** The LSDNSSX1* level shifter is explicitly marked as a transparent component, allowing the CTS engine to "see" through the level shifter to the registers on the other side to balance the skew perfectly across voltage domains. The main backbone is driven by NBUFFX2_LVT.
- **CTS Execution Stages:** The tree is built and routed incrementally using clock_opt -to build_clock and clock_opt -to route_clock.
- **Post-CTS Convergence:** Hold fixing cell options are isolated to high-threshold, low-leakage variants (DELLN*_HVT, NBUFFX2_HVT, NBUFFX4_HVT, NBUFFX8_HVT). The tree is then optimized for high-effort hold timing and clock-aware scan reordering via clock_opt -from final_opto.

**12.3 Issues / violations**

Common violations post-CTS include:

- **Max Skew:** The difference in clock arrival time between the earliest and latest flip-flop is too large.
- **Max Insertion Delay:** The clock takes too long to travel from the port to the flip-flops, making it highly vulnerable to PVT variations.
- **Pulse Width / Duty Cycle Distortion:** The high/low shape of the clock wave degrades as it travels through unequal rise/fall buffer paths.

**12.4 Fixes for Violations**

To fix clock violations without ruining the tree, we apply specific CTS constraints:

- **NDR Usage:** Enforcing strict double-spacing rules on clock nets to shield them from neighbouring data lines.
- **Restricted Cell Lists:** Forcing the tool to use only specific, highly symmetrical clock buffers and inverters rather than standard logic buffers.
- **Manual Guide Buffers:** Manually dropping a strong clock buffer at a divergence point to force the tool to balance a stubborn branch.

**12.5 Fixes for Hold / setup**

Once the clock is "real," it takes time to reach the flip-flops, causing massive shifts in timing.

- **Setup Fixes:** Setup paths fail if data arrives too late. The tool fixes this by downsizing data path logic, cloning heavily loaded cells, or utilizing Useful Skew.
- **Hold Fixes:** Hold paths fail if data arrives too fast (racing past the clock). The tool exclusively fixes this by inserting delay buffers (like the DELLN*_HVT cells defined in the script) into the data path to slow the signal down.

**12.6 Sanity checks**

Verify the clock tree structure by ensuring there are no unrouted clock nets, no overlapping clock buffers, and that the clock signal cleanly passes through the newly inserted level shifters when crossing into the 0.75V domain.

**12.7 Reports**

The script automatically dumps comprehensive summaries into the ./report/ directory:

- report_clock_tree -summary -> Analyzes the longest and shortest paths.
- report_timing -delay_type max/min -> Validates post-CTS setup and hold timing.
- report_clock_qor -type <option> -> A comprehensive command to check the Quality of Results for the clock network. To deeply analyze the clock structure, you can specify the following types:
    - area
    - latency
    - power
    - summary
    - balance_groups
    - level
    - robustness
    - timing_aware_local_skew
    - drc_violators
    - local_skew
    - structure

NEXUS_SOC

## Chapter 13: Routing

With the standard cells placed and the clock tree built, the remaining task is to wire all the logic together. The routing engine must physically connect millions of pins using the available metal layers while avoiding congestion, shorts, and physical manufacturing rules.

### 13.1 Detailed Routing Script

**source ./scripts/Routing/routing_script.tcl**

Detailed routing is executed via this master script, which performs **Production Routing & Signoff Optimization**. It operates in three distinct phases:

- **Global Routing:** Divides the chip into grids and plans the general path for every wire, managing overall congestion.
- **Track Assignment:** Assigns those general paths to specific metal tracks (e.g., Metal 3, Metal 4) in the routing channels.
- **Detailed Routing:** Connects the actual standard cell pins, navigating around obstacles, vias, and adjacent wires to resolve all physical DRCs.

The script enforces critical optimizations:

- **Timing-Driven Routing:** Set to true for global, track, and detail stages to ensure the router honors the setup timing constraints defined in the SDC.
- **Crosstalk Aware Routing:** Enabled for global and track stages to minimize noise injection between adjacent parallel wires.
- **SI Analysis:** Enables Signal Integrity (SI) analysis with statistical crosstalk aggregation (route.time.si_enable_analysis).

### 13.2 Sanity checks

#### 13.2.1 check_design -checks pre_route_stage

Before the heavy detailed routing begins, this crucial check ensures the database is clean. It verifies that placement legality holds, the multi-voltage power grid (VDD_DEFAULT, VDD_HIGH and VDD_LP) is fully intact, and the clock tree is fully routed and fixed (set_dont_touch_network -clock_only [get_ports *clk*]).

#### 13.2.2 check_lvs -max_errors 0

Performs Layout Versus Schematic verification to ensure the physical metal connectivity perfectly matches the logical netlist, ensuring zero net-mismatch errors.

### 13.2.3 check_routes

Validates the integrity of the routed database, identifying any incomplete connections or illegal wiring topologies.

### 13.2.4 check_pg_drc -do_not_check_in_hier_shapes_blocks

Checks the thick power metal routes for spacing and width Design Rule Check (DRC) violations, while ignoring blocks to focus strictly on the grid.

## 13.3 Reports

### 13.3.1 check_routability

This command analyzes the routing grids for DRC hotspots. It ensures that the router did not leave open nets (unrouted wires) or short circuits (wires touching) that cannot be resolved in subsequent optimization passes.

## 13.4 Antenna Rule

During plasma etching in manufacturing, long metal wires act like antennas, accumulating static electrical charge. If this charge has nowhere to go, it blows out the delicate thin-oxide gate of the transistor it connects to.

We enforce these protections by sourcing the foundry-provided ruleset:

- **Command:** source ./scripts/Routing/saed32nm_ant_1p9m.tcl

The router detects and fixes Antenna Violations using two automated methods:

- **Layer Hopping:** Breaking the long wire and routing it up to the highest metal layer to interrupt charge accumulation.
- **Antenna Diodes:** Inserting a reverse-biased diode near the transistor gate to bleed the charge safely.

## 13.5 Post-Routing DRC ECO Fixes

When the detail router fails to resolve dense physical violations natively, targeted Engineering Change Orders (ECOs) are manually applied to eradicate the remaining shorts and opens.

### 13.5.1 Eradicating M1 Shorts

For standard cell pins crashing into unmapped power straps on lower layers, M1_shorts_fixes.tcl provides a targeted escalation sequence. This method isolates the exact failures, rips up the bad routing, explicitly nudges the cells out of the collision zones, locks them, and reroutes:

```
Source ./scripts/M1_short_fix/M1_short_fix.tcl
```

**Note:** For extremely stubborn single-cell shorts, the script utilizes move_objects -delta {15.0 15.0} $last_cell to force the cell far away from the local strap congestion before legalization

### 13.5.2 Resolving M1 & Net Spacing (Fat Contact) Issues

When placement congestion causes standard cell boundaries and vias to violate spacing rules beneath the power grid, M1_and_net_spacing_fat_contact_fix.tcl unlocks the logic layout and forces the engine to disperse the congestion.

```
Source ./scripts/post_routing_fixes/M1_and_net_spacing_fat_contact_fix.tcl
```

### 13.5.3 Resolving Open Nets

If the router leaves nets entirely unconnected (Opens), open_nets_fix_post_routing.tcl forces a highly aggressive routing pass dedicated strictly to sealing the gaps:

```
Source ./scripts/post_routing_fixes/ open_nets_fix_post_routing.tcl
```

### 13.6 Optimization & Save

Once routing is completely clean and DRC-free, we run route_opt to perform final logic-gate optimization and wire-length refinement. The finalized physical layout is preserved: save_block -as route_opt_all

- **Command:** save_block -as route_opt_all

NEXUS_SOC

## Chapter 14: Signoff (Extract with your own Methods)

Signoff is the final phase of the ASIC design cycle. The physical database is locked and rigorous checks are performed using specialized signoff tools to guarantee the chip will function correctly after manufacturing.

### 13.1 source ./scripts/signoff_script.tcl (Only an Example, Not Recommended)

The signoff process involves exporting the physical design data for final verification across several domains:

- **Parasitic Extraction (StarRC):** Extract SPEF (Standard Parasitic Exchange Format) using StarRC. This tool extracts the final, highly accurate resistance and capacitance values (RC) of every single routed wire in the 32nm design.
- **Static Timing Analysis (PrimeTime):** Uses the extracted SPEF data to run a final, golden timing check across all MCMM corners. During this phase, we generate images (visual bottleneck histograms and graphical path profiles) using PrimeTime to visually document timing closure. The Multi-Voltage Crypto Core design must have zero setup and zero hold violations across all active domains (VDD at 0.85V, VDDH at 1.05V, and VDDL at 0.78V).
- **Physical Verification (IC Validator / Calibre):**
    - **DRC (Design Rule Check):** Ensures spacing, width, and via rules strictly meet the 32nm foundry manual.
    - **LVS (Layout Versus Schematic):** Extracts the physical shapes back into a netlist and compares it against the golden synthesis netlist to ensure zero missing or shorted connections.
- **IR Drop / EM Signoff (RedHawk / Voltus):** Confirms the power mesh—especially the lowered 0.78V VDDL grid—delivers adequate voltage to all instances under peak switching activity without electromigration failure.

### 14.2 Tapeout

Once all signoff checks report zero violations, the design achieves **Tapeout Readiness**. The final physical shapes are exported as a GDSII stream format file and sent to the foundry for mask creation and silicon fabrication.

## Chapter 15: Sign-off Timing Analysis & ECO Flow

### Phase 1: ICC2 Preparation & Data Export

Before moving to timing sign-off, the physical database must be structurally sound and free of manual routing artifacts.

1. **Final Physical Verification:** Ensure the block is physically clean after any manual GUI routing fixes.

```
check_pg_drc -do_not_check_in_hier_blocks
check_lvs -max_errors 0
check_routes
```

2. **Export the Post-Route Netlist:**

```
write_verilog ./PRIMETIME/inputs/routed_nexus_soc_netlist.v
```

3. **Export Clean Post-Route SDCs:** Export the constraints directly from the routed database to capture the true propagated clock structures. Do not reuse pre-route SDCs.

```
write_sdc -scenarios func.ss_125c -output ./PRIMETIME/inputs/ss_125_clean.sdc
write_sdc -scenarios func.ff_m40c -output ./PRIMETIME/inputs/ff_m40_clean.sdc
```

### Phase 2: Parasitic Extraction (StarRC)

1. Ensure the directory structure is set up:

```
mkdir -p STARRC/{inputs,outputs/{spef,work},scripts}
```

2. Execute the StarRC extraction scripts.
3. Copy the highly accurate, 3D-extracted .spef files (cbest.spef and cworst.spef) to the ./PRIMETIME/inputs/ directory.

### Phase 3: PrimeTime Environment Setup & Single-Scenario Checks

Setting up the PrimeTime scripts correctly is critical to avoid multi-voltage scaling crashes (SLG-303) and unresolved link references (LNK-003).

1. **Configure link_library Properly:**
    - **Rule:** The link_library must **only** contain the primary/default voltage base cells (0.95V), the SRAM, and the necessary level-shifter libraries (both ulvl and dlvl for all VT flavors: LVT, RVT, HVT).
    - **Do NOT** include the secondary voltage base cells (e.g., 0.75V or 0.85V) in the link_library, or the scaling groups will crash.
2. **Configure Scaling Groups:** Define the scaling groups to map the primary 0.95V cells to their respective secondary voltage (0.75V for max, 0.85V for min) variants.

3. **Clean SDC Sourcing:** Source the _clean.sdc files exported from ICC2. Remove any manual hacks like set_propagated_clock or remove_ideal_network from the PT scripts, as the post-route SDC handles this natively.
4. **Validation:** Invoke pt_shell, source func_max.tcl, and verify report_global_timing. Exit the shell completely, restart it, and repeat for func_min.tcl.

**Phase 4: DMSA (Distributed Multi-Scenario Analysis) Sign-off**

Once individual scenarios link cleanly, use a Master Script to run them concurrently and generate a unified ECO file.

1. **Launch DMSA:**

pt_shell -multi_scenario

2. **Execute the Master Manager Script:** This script should allocate cores, create scenarios, source the verified func_max and func_min scripts to the background workers, and perform a unified timing check.
3. **Generate the ICC2 ECO File:**

```
write_changes -format icc2tcl -output ./outputs/Multi_scenario_final_eco.tcl
```

**Phase 5: ICC2 ECO Implementation**

If PrimeTime generates physical changes (like cell sizing or buffer insertion), implement them precisely without disrupting the rest of the layout.

1. Source the DMSA ECO file in icc2_shell:

```
source ./PRIMETIME/outputs/Multi_scenario_final_eco.tcl
```

2. Legalize and Route the ECOs safely:

```
        check_legality
        legalize_placement -incremental -movable_distance 50
        connect_pg_net
        check_legality
route_eco -utilize_dangling_wires true reuse_existing_global_route true
-reroute modified_nets_first_then_others
```

3. Run the Final Verification Checklist:

```
check_lvs -max_errors 0
check_routes
check_pg_drc -do_not_check_in_hier_blocks
save_block -as route_signoff_pristine
```

**Phase 6: Sign-off Realities & Waivers (Golden Rules)**

PrimeTime is the ultimate source of truth. Rely on its SPEF-driven physics over ICC2's pessimistic routing estimates.

- **The 5-Femtosecond Hold Reality:** If PT reports a minuscule hold violation (e.g., ~0.000005 ns), do not attempt to fix it. The library lacks an X1 drive strength cell to add such a tiny delay, and inserting an X2 buffer (intrinsic delay ~15-20ps) will immediately create a setup violation. Waive this as mathematical noise within OCV margins.
- **The Pessimism Trap:** If ICC2 reports dynamic Max Capacitance, Max Transition, or minor Setup/Hold violations, but PrimeTime reports 0 across the board using SPEF extraction, PrimeTime wins. Do not run further optimizations (route_opt) in ICC2 to chase false violations. Write a waiver and proceed to tape-out.

For additional details and a better understanding of the methodology, refer to the documentation provided with the project files.

- **Document Name:** PrimeTime_Waiving_Documentation.pdf
- **Path:** Documentations/PrimeTime_Waiving_Documentation.pdf

**Note:** The document path is relative to the project directory. Please verify the file under your respective project path.

NEXUS
CHRONOMETER
ASTROMETRICS
ASTROLOBE
PHYSICAL DESIGN
NEXUS_SOC

## Chapter 15: Advanced IR Drop

### 15.1 Power Integrity & Dynamic IR-Drop Signoff

Even if Static Timing Analysis reports zero violations, the silicon can still fail if the power delivery network (PDN) collapses under heavy computational load. Because modern ICC2 environments rely on the Ansys RedHawk-SC fusion integration for native analyze_rail commands, missing this integration requires Power Integrity (PI) analysis to be performed as an external signoff step using dedicated standalone tools (e.g., PrimePower, Voltus, or standalone RedHawk).

### 15.2 Introduction & Physics/Theory

When thousands of logic gates in the nexus_soc toggle simultaneously, they pull a massive surge of current through the resistive metal tracks of the PDN. According to Ohm's Law ($V = I \times R$), this current draw causes the local supply voltage to sag. If the 0.75V VDDL supply temporarily drops to 0.68V, the standard cells switch slower, potentially violating setup time limits that were verified under ideal 0.75V conditions.

### 15.3 Exporting the Design for External Analysis

To run power analysis outside of ICC2, the physical and parasitic data must be accurately exported. The external PI tool requires a snapshot of the fully routed design, the parasitic resistances of the power grid, and the switching activity.

```
# Export the physical layout
write_def -version 5.8 -routing -all_blocks
reports/nexus_soc_top.def

# Export the parasitic RC data
write_parasitics -format SPEF -output reports/nexus_soc_top.spef

# Optional: Generate a power report in ICC2 for baseline
estimation
report_power -scenarios [current_scenario] >
reports/icc2_baseline_power.rpt
```

### 15.4 Files / Inputs Required for Signoff

The external Power Integrity team will ingest the following files:

- **DEF (.def):** Contains the physical placement of all standard cells, macros, and the exact metal routing of the power grid.
- **SPEF (.spef):** Contains the exact resistance and capacitance values of the metal routes.

- **.lib (Liberty):** Contains the internal power models and current draw characteristics for the 0.95V and 0.75V standard cells.
- **VCD/FSDB:** A Value Change Dump generated by the verification team running a gate-level simulation of an active encryption payload. This provides the exact switching timestamp of every net for dynamic analysis.

**15.5 Validating the VDDL Topology**

This standalone analysis proves the physical viability of your custom power plan. The PI tool specifically evaluates the VDDL low-power region to ensure the power mesh connecting directly to the inner guard band is delivering uncompromised current. If the manual topology (which intentionally bypassed macro ring automation) is too resistive, the tool will flag massive voltage droop hotspots in the center of the engine.

**15.6 Heatmaps & Signoff Reports**

The external tool will generate two primary deliverables:

- **Dynamic Voltage Drop Heatmap:** A visual color map of the die highlighting areas where the voltage drops below the accepted threshold (e.g., a drop greater than 5%).
- **Electromigration (EM) Reports:** Identifies metal tracks or vias where the current density (J) exceeds foundry limits, which could cause the wire to degrade or melt over time.

**15.7 Fixes for IR-Drop Violations**

If the PI team flags unacceptable voltage drops, fixes must be implemented back in the ICC2 environment via Engineering Change Orders (ECOs):

- **Adding Decap Cells:** Decoupling capacitors act like tiny local batteries. They are inserted into empty spaces near hotspots to supply instantaneous charge during switching bursts.
- **Thickening Local Straps:** Manually dropping additional power vias or widening specific M7/M8 straps over the starving region to lower the resistance (R).
- **Spreading the Logic:** Applying density padding constraints in ICC2 and incrementally refining placement to break up dense clusters of high-drive standard cells.

## Chapter 16: Hard Macro Conversion (Block Abstraction)

Once the nexus_soc_top core has achieved timing closure, clean IR-drop, and zero DRC/LVS violations, it can be "hardened." Hardening means sealing the internal ~76K standard cells and 100 macros, converting the entire block into a single, black-box component (a Hard Macro) to be placed inside a larger System-on-Chip (SoC).

The top-level SoC tools do not need to see your internal gates; they only need to know the physical boundaries, the pin locations, the power rails, and the timing of the input/output ports.

### 16.1 Physical Abstraction (Creating the NDM / FRAM View)

To use the macro in a top-level ICC2 flow, we must generate a lightweight physical abstraction. This view hides the internal standard cells and only exposes the core boundary, port geometries, and routing blockages (to prevent the top-level router from accidentally dropping vias into your internal logic).

***** Example:**

```
# Open the fully routed and signed-off block
open_block nexus_soc_top_routed

# Create the abstract view (Frame) for top-level hierarchical design
create_abstract -read_only
```

This command generates an abstract NDM view. It identifies which metal layers are saturated by the core and creates placement/routing blockages, leaving only the highest empty metal layers available for top-level "over-the-cell" routing.

### 16.2 Timing Abstraction (Extracted Timing Model - ETM)

Loading internal gates into the top-level Static Timing Analysis (STA) would crash the tool or take days to run. Instead, we generate an Extracted Timing Model (ETM), which reduces the entire block's timing to just the setup/hold requirements of the input pins and the clock-to-q delays of the output pins.

This is typically done in PrimeTime:

```
# Inside PrimeTime
read_verilog nexus_soc_top_routed.v
read_parasitics nexus_soc_top.spef
# ... load constraints and link ...

# Extract the timing model to generate a liberty file
extract_model -format {lib db} -output nexus_soc_top_macro
```

This creates nexus_soc_top_macro.lib and .db. The SoC synthesis and timing teams will use these exactly like they use standard cell libraries.

**16.3 Multi-Voltage & Power Pin Promotion**

Because this is a dual-voltage macro, the top-level SoC must provide both 0.95V and 0.75V to your block. During the abstraction process, your internal power meshes must be promoted as logical Supply Ports.

- The top-level floorplanner will look at your .lef or NDM abstract to find the exact X,Y coordinates of your VDDL guard band mesh.
- The top-level Power Delivery Network (PDN) will then drop massive power vias directly onto those exposed geometries to feed your computational engine.

**16.4 Exporting Industry-Standard Handoff Files**

If the SoC is being assembled using third-party tools (not strictly ICC2), you must export standard interchange formats rather than just an NDM. A complete "Hard Macro IP Kit" consists of four files:

```
# 1. GDSII: The complete, flattened physical layout for manufacturing
write_gds -design nexus_soc_top_routed
./export/nexus_soc_top.gds

# 2. LEF (Library Exchange Format): The physical boundary and pin locations
write_lef ./export/nexus_soc_top.lef

# 3. Verilog (Black-box): A structural shell for top-level LVS

# (Contains only the module declaration and port list, no internal logic)
write_verilog -compress false -hierarchy abstract
./export/nexus_soc_top_bb.v
```

**The Final Hard Macro Deliverables:**

- nexus_soc_top.gds (Manufacturing shapes)
- nexus_soc_top.lef (Physical pins and blockages for the top-level router)
- nexus_soc_top.lib / .db (Timing models for the top-level STA)
- nexus_soc_top_bb.v (Empty wrapper for LVS)
- nexus_soc_top.upf (The power intent, declaring the dual VDD/VDDL pins)



## Chapter 17: Formal Verification (Logic Equivalence Checking)

Even a single flipped gate during placement optimization or a botched routing connection can completely break the functionality of the nexus_soc. Formal Verification uses mathematical proofs to guarantee that the final, routed netlist (with all its inserted clock buffers, level shifters, and scan chains) is 100% logically identical to your original Verilog RTL.

**Important Tool Requirement:** Formal verification cannot be executed inside the icc2_shell. It requires a dedicated external equivalence checking tool like **Synopsys Formality** or **Cadence Conformal**.

### 16.1 Execution via Synopsys Formality

The flow compares a "Reference" (your golden RTL) against an "Implementation" (your routed netlist).

**Tcl (Formality Script):**

Tcl

```
# 1. Setup Reference (Golden RTL)
read_sverilog -container r {./inputs/sram_stub.v ./inputs/nexus_soc_top_sfo.v}
read_db -container r {./inputs/libs/saed32_lv.db ./inputs/libs/saed32_hv.db}
set_top r:/WORK/nexus_soc_top

# 2. Setup Implementation (Routed Netlist from ICC2)
read_verilog -container i ./export/nexus_soc_top_bb.v
read_db -container i {./inputs/libs/saed32_lv.db ./inputs/libs/saed32_hv.db}
read_upf -container i ./outputs/nexus_soc_top.upf
set_top i:/WORK/nexus_soc_top

# 3. Match and Verify
match
verify
```

NEXUS_SOC

## Chapter 18: Design for Manufacturing (DFM)

Just because a design routes cleanly and passes timing does not mean the foundry can manufacture it reliably. DFM techniques are applied in ICC2 just before exporting the final GDSII to maximize silicon yield and prevent microscopic manufacturing defects.

### 18.1 Metal Fill (Dummy Fill)

During manufacturing, chemical-mechanical polishing (CMP) relies on a uniform density of metal across the die. If there are large empty spaces between the routing tracks, the polishing pad will "dish" into the silicon, destroying adjacent wires. We insert electrically floating "dummy" metal shapes to ensure uniform density.

**Important Tool Requirement:** Running native metal fill (signoff_create_metal_fill) inside ICC2 strictly requires the **IC Validator** executable and fusion integration to be present in your environment. If your environment lacks IC Validator, this step is typically deferred and handled by the backend CAD team using standalone signoff tools.

### 18.2 Redundant Via Insertion (Via Optimization)

A single via connecting Metal 3 to Metal 4 can easily fail during fabrication due to dust or misalignment. Wherever there is physical space, we instruct ICC2 to drop a second, redundant via next to the first one. If one fails, the other keeps the connection alive.

Unlike metal fill, this can be executed natively in ICC2 without external validator tools.

**Tcl (ICC2):**

```
Tcl
# Enable extra via insertion prioritizing a low-to-high layer order
set_app_options -name route.detail.insert_redundant_vias_layer_order_low_to_high -value true

# Run incremental route optimization to apply the redundant vias safely
route_opt -incremental
```

NEXUS_SOC

## Chapter 19: The ECO Flow & Final Handoff

What happens if the verification team finds a critical bug in the RTL *after* you have finished Signoff? You cannot afford to rerun the entire multi-day flow from Chapter 4. Instead, we use the Engineering Change Order (ECO) flow to freeze the silicon, freeze the routing, and surgically swap out or insert a few specific gates and patch the local routing without destroying your timing closure.

### 19.1 Functional Metal-Only ECO (Pre-Mask)

If a logic bug is found late, we try to fix it using "spare cells" (floating, unused logic gates intentionally scattered across the floorplan earlier in the flow). By only changing the metal routing to connect a spare cell and leaving the base silicon layers untouched, we save massive mask remanufacturing costs.

**Tcl (ICC2):**

```
Tcl
# Swap connections to an available spare cell
disconnect_net [get_nets broken_logic_net] [get_pins bad_cell/Y]
connect_net [get_nets broken_logic_net] [get_pins spare_inv/Y]

# Reroute ONLY the patched connections
route_eco
```

### 19.2 Final GDSII Handoff

Once all ECOs are applied, DFM is complete, and the design is fully verified, the physical shapes are exported as a GDSII stream format file. This is the final deliverable sent to the foundry for mask creation and silicon fabrication.

**Tcl (ICC2):**

```
Tcl
# Export the final flattened layout for manufacturing
write_gds -design nexus_soc_top_routed ./export/nexus_soc_top.gds
```

NEXUS_SOC